

LLM Agents Factory: Retrieval of Domain-Specific LLM Agents

Vitalii Belov*

Sber AI
Moscow, Russian Federation
Moscow Institute of Physics and
Technology
Dolgoprudny, Russian Federation
belov.vitalii@phystech.edu

Elizaveta Kovtun

Sber AI
Moscow, Russian Federation
Skolkovo Institute of Science and
Technology
Moscow, Russian Federation
Elizaveta.Kovtun@skoltech.ru

Artyom Sosedka

Sber AI
Moscow, Russian Federation
National University of Science and
Technology MISIS
Moscow, Russian Federation
m1801239@edu.misis.ru

Artyom Boyarskikh

Sber AI
Moscow, Russian Federation
boyarskikhae@gmail.com

Andrey Sakhovskiy

Sber AI
Moscow, Russian Federation
Skolkovo Institute of Science and
Technology
Moscow, Russian Federation
andrey.sakhovskiy@gmail.com

Semen Budenny

Sber AI
Moscow, Russian Federation
Artificial Intelligence Research
Institute
Moscow, Russian Federation
sanbudenny@sberbank.ru

Abstract

Large language model (LLM) agents improve task performance by decomposing problems into role-specialized behaviors. However, their practical deployment is often limited by the computational cost and instability associated with the on-the-fly agent design for each user request. To address this, we present *LLM Agents Factory*, a retrieval-based framework that constructs domain-specific and Wikipedia-grounded agents on demand using a base of over 20K predetermined agent profiles. Our framework supports two modes: (1) agent profile *retrieval* via semantic search and (2) *distillation* into a compact model fine-tuned for direct agent generation. Experiments on MMLU, BIG-bench, and BIG-bench Hard in a single-agent scenario demonstrate that our retrieval-based agent construction surpasses non-agent baselines in accuracy while matching AutoGen generation quality with a 120B backbone at a substantially lower inference cost. Our work reveals that retrieval from a structured agent repository provides a cost-efficient, accurate, and controllable alternative to dynamic agent generation, responding to the strict demands of industrial applications. We provide the implementation code and the agent base in <https://huggingface.co/frontier-ai/llm-agent-factory>.

CCS Concepts

• Information systems → Information retrieval; • Computing methodologies → Knowledge representation and reasoning.

Keywords

LLM Agents; Retrieval-Augmented Systems; Low-Latency AI Systems

1 Introduction

Recent LLMs [1, 27] show remarkable capabilities across a wide range of tasks, including open-domain question-answering [42], on-demand retrieval [3], and multi-step reasoning [37]. In practical deployments, LLM-powered production systems often operate under strict constraints regarding latency, effectiveness, and controllability [6]. For this reason, LLM agents with explicit specifications are increasingly viewed as a promising approach for a broad spectrum of industry tasks [33]. Such specifications typically encompass a role or persona assignment that constrains the model’s behavioral policy, along with a set of available tools that enable the invocation of external functions.

Despite empirical evidence justifying the advantages of agent-based approaches over standalone LLMs [41], the on-the-fly generation of agents introduces pronounced limitations that can be detrimental in industrial settings. First, dynamic agent construction impairs both interpretability and controllability, since the obtained agent profiles are not based on any task-specific ontologies or formal domain knowledge [43]. Second, the agent creation process can exhibit high volatility in the properties of the generated entities [7] because of the stochastic nature of LLM inference. Such instability in the characteristics of the resulting agents undermines reproducible operation. Finally, agent instantiation typically depends on long and complex meta-prompts [18], which define roles, tools, formatting requirements, and other agent parameters. This dependency increases token consumption and, consequently, the operational costs. Therefore, these challenges indicate that the dynamic generation of agents presents critical obstacles for real-world systems.

In this work, we treat agent construction as an information retrieval (IR) problem. The proposed framework, called the *LLM Agents Factory*, involves retrieving an agent with a domain-grounded profile from a predefined agent base [15, 17]. The principal **contributions** of our approach are a substantial reduction in latency and token consumption, while yielding competitive or even superior accuracy compared to prior approaches. Importantly, the LLM Agents Factory provides a transparent and governable pipeline for agent

* Corresponding author.

To appear in: ACM SIGIR 2026. DOI: <https://doi.org/10.1145/3805712.3808515>.

© Vitalii Belov/ACM 2026. This is the author’s version. The definitive Version of Record will be published in ACM SIGIR 2026 Proceedings.

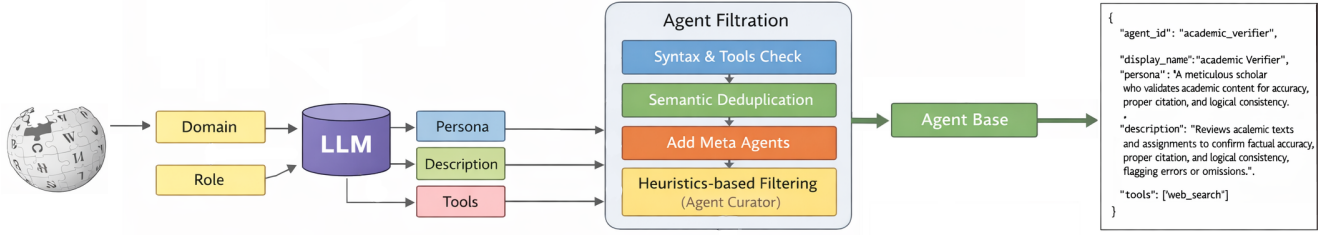


Figure 1: Agent base construction pipeline: from (domain, role) sampling and LLM-based profile generation (persona, description, tools) to multi-stage filtration (syntax and tools checks, semantic deduplication, adding meta agents, and heuristics-based curation), resulting in the final agent base. The right panel shows an example agent profile in JSON.

generation. The specified attributes make our framework beneficial for highly-regulated industrial applications.

2 Related Work

Our work connects agent construction, retrieval, and distillation. We position *agent construction* as an IR problem over a structured repository of agent profiles (personas, constraints, tools), enabling fast selection and low-cost refinement.

Prompting and Structured LLM Inference. LLM behavior can be guided via prompting and instruction-based adaptation, including structured reasoning and sampling for robustness [4, 23, 35, 37]. Tool-augmented LLM systems further improve controllability by exposing search, code, and APIs through structured schemas [25, 26, 29, 40]. However, complex prompting or expensive runtime interventions [16, 35], requiring multiple LLM calls for a single request, may significantly increase inference time and token consumption, prohibiting deployment in real-world systems.

Multi-Agent Orchestration and Automatic Agent Design. Multi-agent LLM systems benefit from role specialization, planning, and critique, with frameworks enabling dynamic agent creation and coordination [18, 24, 39]. However, on-the-fly agent generation can dominate latency and token cost and is sensitive to orchestration details. We share the goal of automatic agent construction but prefer reproducible selection from an ontology-aligned repository, optionally refined with retrieved context. Most existing open-source systems are orchestration toolkits rather than end-to-end solutions that jointly optimize agent construction and execution under cost constraints. Among widely used frameworks, AutoGen is one of the closest to an end-to-end pipeline for dynamic agent creation and execution, but it still relies on long runtime prompting and expensive backbones [39]. Recent work explores end-to-end optimization of tool-using and agentic behaviors with reinforcement learning (e.g., AgentFlow [19] and Search-R1 [13]), but these do not address retrieval over large structured repositories of reusable agent specifications.

Information Retrieval for LLM Inference. *Retrieval Augmented Generation* (RAG) typically performs dense retrieval over an external knowledge source, e.g., a text corpus [15, 17], to dynamically modify the behavior of LLMs and provide it with relevant and up-to-date information. RAG is typically implemented with a lightweight and efficient retriever [8, 17, 20], making the technique vital for LLM applications. In this paper, we combine the agentic and retrieval approaches to enable scalable on-the-fly agent selection.

Further, we adopt synthetic supervision to distill retrieval-based agent generation into a compact model [11, 36].

3 LLM Agents Factory

This section defines the problem statement and describes the key components of *LLM Agents Factory*, a novel retrieval-based framework for agentic LLM inference: (i) Automatic **Agent Base Construction** that provides a reusable agent repository (3.1); (ii) **Agent Library Retrieval (ALR)** that enables rapid agent construction in runtime through the lightweight retrieval module (3.2). Furthermore, we explore the distillation of the ALR results into a compact LLM-based agent generator (3.3).

3.1 Agent Base Construction

Agent Specification. We formalize an LLM agent a as follows: $a = \{\text{base, domain, role, persona, description, tools}\}$, where base is a backbone LLM; domain is an agent’s domain of expertise and role is a discrete behavior label from a manually curated catalog of 40 roles (e.g., *planner*, *verifier*, *tutor*), inspired by recurring role specializations used in prior LLM-agent frameworks [18, 39]; persona and description are natural language descriptions of (i) the agent’s traits and preferences and (ii) the specifications of its functionality; tools is a list of available functions that the agent can call. This formalization extends prior research [29, 39, 40] on agentic systems that did not align LLM agents with the structure of human domain knowledge.

Ontology-Guided Agent Construction. In *LLM Agents Factory*, the construction of the agent base is aimed at two primary goals: (i) reduce the computational cost of LLM agentic systems by pre-generating a comprehensive set of possible agents; (ii) enhance agent interpretability by grounding an agent’s domain to the Wikipedia category system [9, 38]. Specifically, we use 691 high-level Wikipedia categories to serve as domain. We enumerate a 691×40 grid of candidate (domain, role) cells; for each cell, the teacher model attempts to generate up to $K \leq 5$ profiles. Overall, each agent in LLM Agents Factory is constructed through the following steps:

- (1) **Domain Choice** One of 691 domains is randomly selected.
- (2) **Role Assignment:** We sample a role uniformly at random from a fixed catalog of 40 roles (e.g., *verifier*, *planner*), independently of the domain.
- (3) **Profile Generation:** Given a (domain, role) pair, an LLM is prompted to generate up to 5 (persona, description) pairs

Table 1: Main results on MMLU, BIG-bench, and BBH using a fixed solver M (GPT-OSS 120B, $t=0.3$). We report accuracy (Acc), total tokens (TotalTok, in millions), and mean end-to-end latency (TotT, in seconds). Methods are grouped into *baselines* and *our methods* (ALR + distilled generator). For each method and benchmark, we report the best-performing configuration over the evaluation grid (e.g., retrieval hyperparameters such as Top- K and, when applicable, reranking settings); thus, reranking is treated as a configuration option rather than a separate method. Best values per dataset and column are in bold; the second-best values are underlined.

Method	MMLU (N=2,070)			BIG-bench (N=39,185)			BBH (N=2,437)		
	Acc (%)	TotalTok (M)	TotT (s)	Acc (%)	TotalTok (M)	TotT (s)	Acc (%)	TotalTok (M)	TotT (s)
<i>Baselines</i>									
Non-agent	81.9	0.7M	2.09	84.7	16.7M	2.60	68.5	1.0M	2.94
Qwen3-4B zero-shot	81.2	2.3M	2.04	84.3	48.3M	2.68	68.4	2.8M	2.76
AutoGen	80.9	2.4M	6.55	83.4	47.0M	5.25	64.9	2.9M	6.49
<i>ALR, single-agent</i>									
ALR	82.3	<u>0.8M</u>	1.62	<u>85.6</u>	<u>17.2M</u>	2.10	68.5	<u>1.0M</u>	2.10
ALR-Distill (fine-tuned Qwen3-4B)	<u>82.0</u>	1.6M	<u>1.77</u>	85.7	33.4M	<u>2.47</u>	69.3	2.0M	<u>2.59</u>
<i>ALR, multi-agent</i>									
ALR Top-K	82.3	2.3M	5.01	84.4	44.8M	4.55	69.1	2.8M	8.42
ALR + Qwen3-4B zero-shot	82.3	2.6M	2.84	84.9	51.7M	3.15	<u>69.5</u>	3.1M	2.96
ALR + ALR-Distill	82.3	1.8M	2.15	85.1	37.1M	3.02	69.6	2.2M	3.15

each defining a separate agent. The model is allowed to return an empty set for an illogical pair. At this step, we generate the agent specification as JSON using GPT-OSS 120B [1].

- (4) **Tool Selection:** We restrict each profile’s tools field to a fixed whitelist of 10 executable tools derived from the OpenAI tool-calling interface and tool specifications [21, 22]. This constraint ensures the produced agents are executable.

Figure 1 illustrates the agent base construction and filtration pipeline; the right panel shows an example resulting agent profile. Domain and role constraints are injected as hard prompts during generation, anchoring each profile to its ontological cell and preventing category drift. This Cartesian product approach yields an initial pool of over 20K candidate profiles.

Agent Filtering. To ensure agent executability and diversity, candidates undergo a three-stage filtration pipeline 1:

- **Syntax & Tools:** Profiles must parse as valid JSON and restrict tools to a static whitelist of 10 core functions.
- **Semantic Deduplication:** Profiles are embedded using all-mpnet-base-v2. Near-duplicates within domains are identified via FAISS [14] cosine search and merged using a union-find algorithm.
- **Meta Agents Addition:** We add a small set of manually curated, domain-agnostic ‘meta’ profiles (e.g., generic router, triage, and verifier) to improve coverage and provide safe fallbacks for out-of-domain or underspecified queries.
- **Heuristics-based filtering:** A final validation step removes redundant or low-quality profiles based on role-specific heuristics.

This process removes approximately 18% of generated candidates, resulting in higher quality agents.

3.2 Agent Library Retrieval (ALR)

To enable the use of the most relevant agents from LLM Agents Factory in run-time, we introduce **ALR**, a lightweight method for agent retrieval conditioned on a user query.

Agent Retrieval. We formulate agent selection as a dense retrieval task over the structured agent corpus $\mathcal{A}$. **Indexing.** Each agent profile a is serialized into a flat text representation $\text{ser}(a)$ by concatenating its fields (domain, role, persona, description, tools) with explicit separators. We compute embeddings $\mathbf{h}_a = f(\text{ser}(a))$ using a Sentence-BERT bi-encoder $f(\cdot)$ [28].

Given a user query q , we compute the query embedding $\mathbf{h}_q = f(q)$ and retrieve the Top- K agents based on cosine similarity:

$$\text{TopK}(q) = \arg \text{top } k \left(\mathbf{h}_q^\top \mathbf{h}_a \right)_{a \in \mathcal{A}} \quad (1)$$

3.3 ALR-Distill

As an alternative to latency overhead from index search and context injection caused by retrieval, we propose **ALR-Distill**, a supervision-based method to distill the LLM Agents Factory’s agent base into a compact generator model.

Training Data. We construct the training dataset for the agent generation model as follows. For each agent $a \in \mathcal{A}$, we synthesize $T=11$ distinct user requests $\{q_j\}_{j=1}^T$ using GPT-OSS 120B prompted to generate a valid user request given all agent attributes. This results in 220K $((q, a))$ query-agent pairs where q is an input and a is a ground truth agent description in JSON format.

Training Objective. We fine-tune the student model on the synthetic query-agent pairs constructed in Section 3.1. The model is trained to directly generate the JSON agent profile a given the user query q , maximizing the likelihood $P(a|q)$. This allows the system to bypass explicit retrieval at inference time.

Retrieval-Augmented Distillation. We also explore a hybrid regime where the student model conditions on retrieved candidates. In this setup, the Top- K profiles from the bi-encoder are provided as context, and the student model is trained to refine or select the optimal profile rather than generating from scratch. This approach leverages the retrieval index as a soft constraint, reducing the generation search space while maintaining the low-latency benefits of a small model. Both variants enable cost-effective agent construction compared to dynamic generation with large backbones.

3.4 ALR Setups

Using the *LLM Agents Factory*, we consider two agentic usage scenarios: (i) **single-agent**, where the chosen agent profile a is applied to backbone LLM, e.g., persona and description are explicitly in an LLM’s system prompt; (ii) **multi-agent** case, where the model is prompted with a set of K profiles: $A = \{a_1, \dots, a_K\}$ to make inference using its chosen profile $a \in A$. For the *single-agent* case, we use two approaches: retrieval-based **ALR** and fine-tuning-based **ALR-Distill**. For the *multi-agent* case, we propose two approaches:

- **ALR Top- K** : retrieves Top- K agents (e.g. $K=1..10$) passed to the solver LLM as a structured context.
- **ALR-RAG + Qwen generator**: prompts an LLM with two profiles (a, a') , each obtained via the *single-agent* approach: (i) a from *ALR* and (ii) a' generated by Qwen (either in a zero-shot or supervised ALR-Distill setting).

4 Experiments

4.1 Experimental Setup

Baselines. We fix the solver backbone M to GPT-OSS 120B [1] for all experiments and vary only the agent construction mechanism.

- **Non-agent**: a solver LLM receives only the task prompt without any agent specification.
- **Qwen3-4B Zero-shot**: a compact Qwen3-4B LLM generates an agent profile from the user request.
- **AutoGen** [39]: agent profile fields are generated at runtime using the same role, persona, description, and tool schema as in our repository, while the solver backbone and decoding settings are kept fixed.

Evaluation Data. In order to evaluate our methods against baselines and various experimental setups, we chose three benchmarks covering diverse domains and levels of difficulty, namely MMLU [10], BIG-bench [31], and BIG-bench Hard (BBH) [32].

Evaluation Setup. We use accuracy as the primary metric to assess quality in downstream tasks. To assess efficiency, we measure: (i) overall token consumption for all inference steps, including agent construction and solver inference, summed across all dataset samples; (ii) mean end-to-end latency, in seconds, measured from receiving the request to producing the final answer.

4.2 Implementation Details

Agent Retrieval. For the bi-encoder, we adopt several sentence embedding models, including small- and base-sized BGE [5], MiniLM [34], and MPNet [30] encoders. MPNet consistently performs best in

our preliminary experiments, showing higher downstream accuracy compared to other tested encoders. Thus, this model is used for agent retrieval in all reported results. The embeddings are L2-normalized and indexed with FAISS [14] (inner-product search), which corresponds to cosine similarity.

ALR Setup. For *ALR Top- K* , we set the number of retrieved profiles to $K = 5$. For *ALR-Distill*, we fine-tune Qwen3-4B for 5 epochs using a sequence length of 2048, cosine learning rate schedule with 50 warmup steps and mixed precision. The model is trained via Low-Rank Adaptation (LoRA) [12] with hyperparameters $\text{learning_rate} \approx 2.43 \times 10^{-4}$, $r=32$, $\alpha=128$, and $\text{dropout}=0.1$ optimized using Optuna [2].

4.3 Artifacts and Reproducibility

We release the implementation code and agent base to support reproducibility and practical adoption of the proposed framework.¹ The artifact includes the agent database, domain, role, and tool configuration files, retrieval modules, RAG-based generation utilities, command-line interfaces, and tests. The repository supports both direct agent search and retrieval-augmented agent generation, enabling users to inspect retrieved profiles, reproduce the agent selection pipeline, and adapt the system to new deployment settings.

5 Results

The evaluation results are presented in Table 1. We draw the following key observations.

Agent Retrieval is Effective and Efficient. Compared to AutoGen, which relies on runtime profile generation, retrieval-based ALR shows consistently higher accuracy while introducing minor token and runtime overhead compared to the non-agent baseline. Specifically, our method consumes $\sim 3\times$ fewer tokens across all datasets and shows faster inference. On MMLU and BIG-bench, ALR exceeds non-agent LLM inference, indicating that the approach produces more task-relevant agent specifications. This suggests that runtime profile generation may introduce unnecessary complexity for standard question-answering and reasoning tasks.

Retrieval vs. Fine-Tuning Trade-Off. For the *single-agent* setup, the fine-tuned ALR-Distill performs on par with retrieval-based ALR while doubling token consumption and marginally increasing latency. The retrieval-based approach is more favorable on our LLM Agents Factory for quality–latency balance. Presumably, the preference could lean towards distillation for a significantly larger agent base, providing a deployment-friendly alternative.

Single-Agent vs. Multi-Agent Trade-Off. From our results, the relative effectiveness of single-agent and multi-agent setups depends on task complexity. On MMLU and BIG-bench, single-agent setups (ALR and ALR-Distill) achieve the highest accuracy (82.3% and 85.7%, respectively) while two-agent approaches perform better on the harder BBH dataset. However, the improvement comes at the cost of notable token consumption and an increase in runtime.

¹<https://huggingface.co/frontier-ai/llm-agent-factory>

6 Discussion and Limitations

Our evaluation isolates the effect of agent construction by fixing the downstream solver and varying only the mechanism used to obtain an agent profile. Therefore, the reported results should be interpreted as evidence for efficient agent specification selection on standard question-answering and reasoning benchmarks, rather than as a complete evaluation of long-horizon, multi-turn, or tool-intensive agentic behavior.

The AutoGen baseline follows the same profile schema used in our repository: role, persona, description, and tool fields are generated at runtime, while the solver backbone and decoding settings are kept fixed. This setup focuses the comparison on the central design choice of this work: runtime profile generation versus retrieval from a predefined and validated agent repository. The reported accuracy differences were verified as statistically significant using z-tests.

Meta agents are included in the same retrieval index as all domain-grounded profiles and are not treated as a separate evaluation method. They serve as fallback profiles for underspecified or out-of-domain queries and are selected only if their retrieval score exceeds that of specialized profiles. In the reported benchmark setting, the observed gains therefore reflect the behavior of the full repository-based selection mechanism rather than a manually triggered fallback policy.

The current repository is grounded in Wikipedia categories and a fixed catalog of roles. This design improves interpretability and reproducibility, but it can underrepresent niche industrial domains and proprietary taxonomies. In practical deployments, the same construction pipeline can be instantiated over domain-specific ontologies, including medical, legal, financial, or enterprise schemas.

Both the profile base and the distillation data rely on synthetic supervision from a strong teacher model. Biases, omissions, or hallucinated assumptions of the teacher may therefore propagate into the generated profiles and query-agent pairs. We mitigate this risk through schema validation, tool whitelisting, semantic deduplication, and heuristic filtering, but domain-expert validation remains important for high-stakes industrial use cases.

Future work will extend ALR with metadata-aware retrieval over explicit schema fields such as domain, role, and tools, and with hierarchical orchestration that invokes multi-agent reasoning only when task complexity justifies the additional cost. Another direction is to refine the distillation objective toward concise functional JSON profiles, reducing the token overhead of generated agents while preserving their task-specific behavior.

7 Conclusion

We present LLM Agents Factory, a retrieval-based framework that treats agent construction as an information retrieval problem over a structured repository of model-agnostic agents grounded in the Wikipedia taxonomy. Experiments across MMLU, BIG-bench, and BBH demonstrate that lightweight retrieval mechanisms can surpass dynamic agent generation while reducing token consumption and latency by up to 3x and 4x, respectively. Our work addresses key limitations of runtime agent generation for industrial LLM systems: it improves interpretability through structured domain and role labels, improves reproducibility through deterministic retrieval

over a validated agent base, and substantially reduces the token overhead introduced by long orchestration prompts.

References

- [1] Sandhini Agarwal et al. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv preprint arXiv:2508.10925* (2025). doi:10.48550/arXiv.2508.10925
- [2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-Generation Hyperparameter Optimization Framework. In *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2623–2631.
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2005.14165>
- [5] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2318–2335. doi:10.18653/v1/2024.findings-acl.137
- [6] Sophie Fischer, Carlos Gemmell, Niklas Tecklenburg, Iain Mackie, Federico Rossetto, and Jeffrey Dalton. 2024. GRILLBot in practice: Lessons and tradeoffs deploying large language models for adaptable conversational task assistants. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4951–4961.
- [7] Fabiana Fournier, Lior Limonad, and Yuval David. 2025. Agentic Process Observability: Discovering Behavioral Variability. In *European Conference on Artificial Intelligence*.
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)*. JMLR.org, Article 368, 10 pages.
- [9] Nicolas Heist and Heiko Paulheim. 2019. Uncovering the Semantics of Wikipedia Categories. *arXiv preprint arXiv:1906.12089* (2019). doi:10.48550/arXiv.1906.12089
- [10] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2009.03300>
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015). <https://arxiv.org/abs/1503.02531>
- [12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net. <https://openreview.net/forum?id=nZvKeeFYf9>
- [13] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *arXiv preprint arXiv:2503.09516* (2025). <https://arxiv.org/abs/2503.09516>
- [14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [15] Vladimir Karpukhin, Barlas Ögüz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. doi:10.18653/v1/2020.emnlp-main.550
- [16] Akbair Khan, John Hughes, Dan Valentine, Laura Ruis, Kshitij Sachan, Ansh Radhakrishnan, Edward Grefenstette, Samuel R. Bowman, Tim Rocktäschel, and Ethan Perez. 2024. Debating with More Persuasive LLMs Leads to More Truthful Answers. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024 (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR / OpenReview.net, 23662–23733. <https://proceedings.mlr.press/v235/khan24a.html>
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2005.11401>

- [18] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. *arXiv preprint arXiv:2303.17760* (2023). <https://arxiv.org/abs/2303.17760>
- [19] Zhuofeng Li, Haoxiang Zhang, Seungju Han, Sheng Liu, Jianwen Xie, Yu Zhang, Yejin Choi, James Zou, and Pan Lu. 2026. In-the-Flow Agentic System Optimization for Effective Planning and Tool Use. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2510.05592>
- [20] Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FACTScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 12076–12100. doi:10.18653/v1/2023.emnlp-main.741
- [21] OpenAI. 2025. Function calling. <https://developers.openai.com/api/docs/guides/function-calling/>. Accessed: 2026-02-26.
- [22] OpenAI. 2025. Using tools. <https://developers.openai.com/api/docs/guides/tools/>. Accessed: 2026-02-26.
- [23] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2203.02155>
- [24] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*. doi:10.1145/3586183.3606763
- [25] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334* (2023). <https://arxiv.org/abs/2305.15334>
- [26] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023). <https://arxiv.org/abs/2307.16789>
- [27] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025). <https://arxiv.org/abs/2505.09388>
- [28] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410
- [29] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761* (2023). <https://arxiv.org/abs/2302.04761>
- [30] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. MPNet: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://proceedings.neurips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html>
- [31] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615* (2022). <https://arxiv.org/abs/2206.04615>
- [32] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261* (2022). <https://arxiv.org/abs/2210.09261>
- [33] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [34] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [35] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=1PL1NIMMrw>
- [36] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. doi:10.18653/v1/2023.acl-long.754
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2201.11903>
- [38] Wikipedia contributors. 2026. Wikipedia: Categorization. <https://en.wikipedia.org/wiki/Wikipedia:Categorization>. Accessed: 2026-02-25.
- [39] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155* (2023). <https://arxiv.org/abs/2308.08155>
- [40] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2210.03629>
- [41] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- [42] Wenhao Yu, Dan Iter, Shuohang Wang, Yichong Xu, Mingxuan Ju, Soumya Sanyal, Chenguang Zhu, Michael Zeng, and Meng Jiang. [n.d.]. Generate rather than Retrieve: Large Language Models are Strong Context Generators. In *The Eleventh International Conference on Learning Representations*.
- [43] Xiaochi Zhou, Patrick Bulter, Changxuan Yang, Simon D Rihm, Thitikarn Angkanaporn, Jethro Akroyd, Sebastian Mosbach, and Markus Kraft. 2026. Ontology-to-tools compilation for executable semantic constraint enforcement in LLM agents. *arXiv preprint arXiv:2602.03439* (2026).